



ADDIS ABABA
**SCIENCE AND
TECHNOLOGY**
UNIVERSITY
UNIVERSITY FOR INDUSTRY

**Smart Real-Time Driver State Monitoring and Emergency
Detection System**
Embedded Systems

Group Members

Name	ID
Tesfamichael Tafere	ETS 1303/15
Tewodros Gebretsadik	ETS 1305/15
Thressa Menilek	ETS 1309/15
Tigist Kinfemichael	ETS 1310/15
Tsegaye Berhe	ETS 1325/15

Submitted to: Instructor Kassahun Tadesse

Date: April 13, 2026

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Current Progress
5. System Architecture
6. Components Used (Virtual & Software)
7. System Logic
8. Implementation Plan
9. Challenges
10. Next Steps
11. Conclusion



1. Introduction

This project focuses on creating a Smart Real-Time Driver State Monitoring and Emergency Detection System. This embedded project integrates two important fields of vehicle safety: preventive measures in case of potential hazards and an emergency solution in case of impact. The embedded project will be a purely virtual demonstration using software-based algorithms and Proteus rather than building hardware.

2. Problem Statement

Many injuries result from road traffic accidents globally, and most accidents happen because of the driver's inattentiveness and inability to react in time due to fatigue, brief sleep periods, or distraction. Moreover, when the accident happens, it is not always easy for the victim to get help fast if the person is alone, injured, or in the area that has no people nearby. Most outdated vehicles lack sensors to warn drivers. Available monitoring tools are generally too expensive, commercial and hard to install. In addition, usually drowsiness monitoring and crash notification are separate products.

3. Objectives

- To create an intrusive embedded system that detects drowsiness and distracted driving based on the computer vision technology;
- Implement the driver's drowsiness detection algorithm based on Eye Aspect Ratio (EAR) and other features (blink persistence, mouth opening and head-posture);

- Create a subsystem capable of detecting sudden G-force changes associated with a crash event (based on a virtual MPU6050);
- Design the communication subsystem able to handle GPS coordinate acquisition and GSM-based emergency SMS;
- Create virtual embedded environment in Proteus.

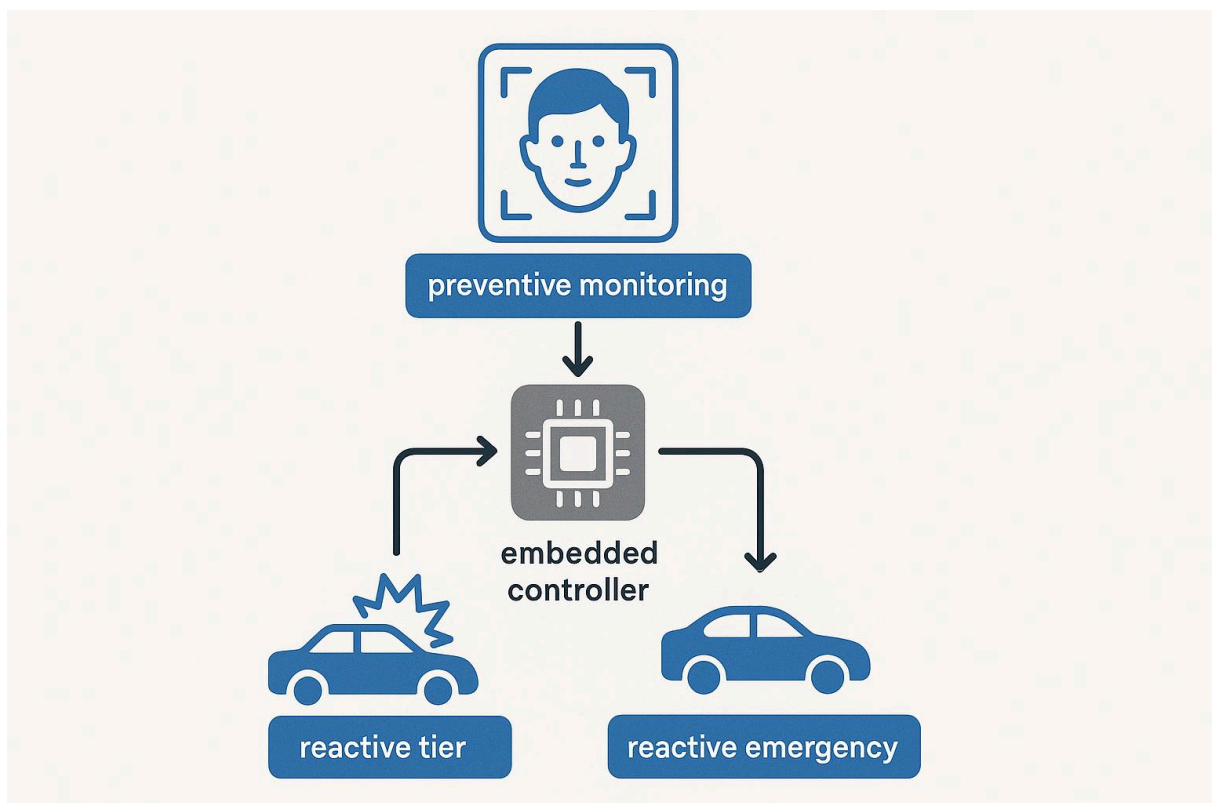
4. Current Progress

By now we managed to finish all requirements analysis and literature review stage. Functional requirements and system thresholds have been defined. The design scope has been approved with our decision to go with a purely virtual simulation approach rather than assembling physical prototype on a breadboard. We've finished preliminary research on landmark detection approaches and serial communication methods.

5. System Architecture

The project uses Two Tiered Safety Approach:

- **Tier 1 (Preventive):** Continuous evaluation of driver's face, eye activities and posture in order to determine distraction or drowsiness. In case any signs are detected, local warnings (LED lights, messages on LCD screen and buzzers) are triggered.
- **Tier 2 (Reactive):** Crash subsystem that detects the occurrence of a collision. In case a crash is detected, it escalates to emergency mode and prepares an SOS message with the current coordinates of the vehicle.



Virtual architecture separates the software layer from the hardware layer. Python script runs vision code externally, and MCU works on a simulated circuit in Proteus. Virtual COM-port or some kind of serial emulator would be used as an intermediate between the outputs of computer vision and simulation of embedded system in Proteus.

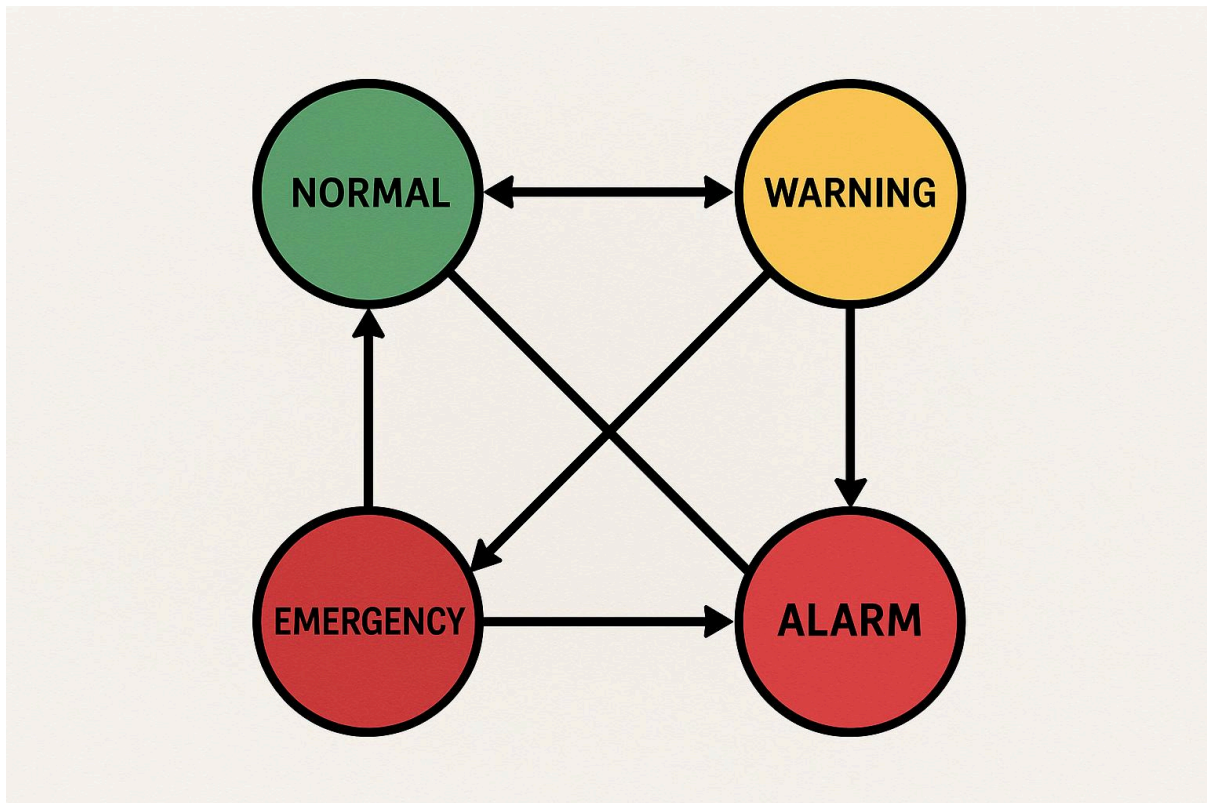
6. Components Used (Virtual & Software)

- **Arduino / ESP32 compatible MCU** – embedded controller simulation;
- **PC runtime (Python/OpenCV)** – software that performs vision analysis;
- **USB webcam or dataset of videos** for face detection;
- **Virtual MPU6050** – simulation of crash detection;
- **Virtual GPS & GSM (NEO-6M / SIM800L)** – simulation of SMS with coordinates generation;
- **Proteus** – virtual hardware design.

7. System Logic

The state machine concept was selected to describe the state transitions (normal, warning, alarm and emergency).

- **Vision State Machine Logic:** Each frame is analyzed, and if EAR drops below threshold within a certain period of time – drowsiness state is triggered and sent to the MCU.
- **Priority Scheme:** The system has priority logic which prevents computing heavy tasks from interfering with emergency task execution. In case there is an MPU6050 spike, then emergency task gets highest priority and triggers an SOS message.



8. Implementation Plan

1. **Phase 2:** Implementing the EAR, yawning and head-pose detection algorithm in Python;
2. **Phase 3:** Creating a virtual schematic in Proteus, setting up the virtual MCU, connecting output components such as LCD, buzzers and LED lights;
3. **Phase 4:** Creating a GPS & GSM serial simulation for emergency message generation;
4. **Phase 5:** Setting up the bridge between Python code and virtual MCU in Proteus and running crash/drowsiness test scenarios.

9. Challenges

- **Integration issues:** Creating a stable and working communication between Python computer vision code and simulated hardware in Proteus.
- **Vision accuracy:** Maintaining stable landmarks in low light condition or under bad video quality.
- **Simulation fidelity:** Simulating real inputs of virtual MPU6050 in order to detect G-force spike accurately enough without conducting any physical testing.

10. Next Steps

- Finish the implementation of the EAR and head pose detection algorithm locally on a PC;

- Determine the preliminary blink persistence and mouth opening threshold;
- Start working on Proteus schematic creation;
- Set up a virtual COM-port communication between Python script and a terminal.

11. Conclusion

We managed to define the foundation of the project which will be further demonstrated in Proteus. We will not require any special purchases and will be able to focus only on developing an algorithmically correct project. From now on, we need to pay special attention to implementing our vision thresholding correctly and ensuring reliable communication between our software and simulation of the embedded environment.